

DozerNet — Team Setup Guide

We now have one shared GitHub repo with a branch per person, each starting from the same empty scaffold. You clone it, check out your own branch, and build your module from scratch there — your own files, your own commit history. No copying files between people.

Your repo (push here): <https://github.com/IT25102642/DozerNet>

Reference repo (view/copy from here): <https://github.com/ananian04/DozerNet> — branch main has a finished, working version of every module. Open the file at the path listed for your module (Part B), copy its contents, and create a new file at the same path in your own branch of your repo. Do this from your own machine so it becomes your own commit.

#	Module	Owner	Branch	Status
1	Customer Management	G. Ananian	feature/m1-customer-management	✓ started
2	Booking & Rental Management	Theekshana A.D.	feature/m2-booking-rental	not started
3	Fleet (JCB) Management	Jayawardhana A.G.D.L.	feature/m3-fleet-management	not started
4	Operator Management	Perera M.A.S.A.	feature/m4-operator-management	not started
5	Maintenance & Service Management	Ruvendi R.G.K.	feature/m5-maintenance-service	not started
6	Payment & Administration	Thathsara J.P.T.	feature/m6-payment-admin	not started

Important — read this before you start writing code. Every branch currently has only `pom.xml`, `.gitignore`, and empty `module/template` folders. There is **no shared common/ package yet** (no `User/Role` model, no security config, no notification service, no exception handling) and **no Spring Boot entry point** (`DozerNetApplication.java`). That means **the app will not compile or run yet** on any branch, including yours. You can still write your module's classes and templates — just know you can't build/run the full app locally until the team has agreed who adds the shared `common/` layer back, and it's merged into every branch. Talk to your team lead about this before you're blocked.

PART A — One-time setup (everyone does this)

A1. Install these first

- **JDK 21** (not newer — check with `java -version`).
- **Maven 3.9+** — check with `mvn -version`.
- **MySQL 8** (needed later, once the app can run — see A5).
- **Git**, and your GitHub account added as a collaborator on the repo above.
- An IDE (IntelliJ IDEA Community is fine) — open the project as a Maven project.

A2. Clone the repo and check out your own branch

Everyone clones the **same** repo, then switches to their own branch — never work directly on main.

```
git clone https://github.com/IT25102642/DozerNet
cd DozerNet
git checkout feature/mN-your-module # use YOUR branch name from the table above
```

A3. What's already there, and what's yours to build

Two empty folders are waiting for you — everything inside them is yours to create:

```
src/main/java/com/dozernet/moduleN_xxx/ <- your Java package (entity, repository,
service, web, dto sub-folders – organise
it the way the rest of the app does)
src/main/resources/templates/<your-folder>/ <- your Thymeleaf pages
```

Only add or edit files inside those two folders. If you need something in common/ (a shared entity, a security rule, a new notification type) that doesn't exist yet, raise it with the team first rather than adding it yourself — two people editing the same shared file on different branches at the same time is exactly what causes merge conflicts later.

A4. Commit in logical, reasonable chunks — not one giant commit

Break your work into commits that each do one coherent thing, with a clear message. Here's the actual history from the Customer Management branch as a worked example:

```
feat(customer): registration
RegisterForm.java, RegistrationController.java, templates/customer/register.html

feat(customer): profile, password change and dashboard
ProfileForm.java, CustomerService.java, CustomerController.java,
templates/customer/dashboard.html, templates/customer/profile.html

feat(customer): notification inbox
CustomerNotificationService.java, CustomerNotificationController.java,
templates/customer/notifications.html
```

Three commits, each a complete, reviewable feature slice — not "WIP", not everything at once. Aim for the same shape: as you finish one working piece of your module, commit it, then move to the next.

```
git add <the files for this piece>
git commit -m "feat(yourmodule): short description of this piece"
git push origin feature/mN-your-module # pushes to YOUR branch only
```

Never `git push origin main` yourself. Once your module works, open a Pull Request from your branch into main so the team can review it before it merges (see A6).

A5. Set up MySQL (for once the app can run)

1. Start MySQL, then create the database:

```
mysql -u root -p
CREATE DATABASE dozernet;
EXIT;
```

2. Once `application-mysql.properties` exists (part of the shared common/config layer), set `spring.datasource.username` / `spring.datasource.password` to your own MySQL login.

3. Run with `mvn spring-boot:run` (H2, zero setup) or `mvn spring-boot:run -Dspring-boot.run.profiles=mysql` once that's all in place.

A6. Merging back into main

When your module is ready (and the shared common/ layer exists so it actually compiles), open a Pull Request on GitHub from your branch into main. Don't merge your own PR without at least one other team member looking at it — that's the point of working on separate branches in the first place.

PART B — #1 Customer Management (already started)

G. Ananian · IT25102642

Branch: `feature/m1-customer-management` | Your package: `com.dozernet.module1_customer` | Your templates: `templates/customer/`

This branch already has 3 commits building out registration, profile/dashboard, and the notification inbox — see A4 on the previous pages for the exact commit breakdown. Use it as the reference for how your own commits should look.

Reference: <https://github.com/ananian04/DozerNet> (branch main)

Backend files to view & recreate

Path on the reference repo: `src/main/java/com/dozernet/module1_customer/<path>` — create the same file at `module1_customer/<path>` in your own branch.

- `dto/ProfileForm.java`
- `dto/RegisterForm.java`
- `service/CustomerNotificationService.java`
- `service/CustomerService.java`
- `web/CustomerController.java`
- `web/CustomerNotificationController.java`
- `web/RegistrationController.java`

Frontend files to view & recreate

Path on the reference repo: `src/main/resources/templates/customer/<file>`.

- `dashboard.html`
- `notifications.html`
- `profile.html`
- `register.html`

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.

PART B — #2 Booking & Rental Management

Theekshana A.D. · IT25101810

Branch: `feature/m2-booking-rental` | Your package: `com.dozernet.module2_booking` | Your templates: `templates/booking/`

Nothing has been pushed to `feature/m2-booking-rental` yet beyond the shared scaffold. Open the reference repo below, copy each file's contents, and create it at the matching path in your own branch.

Reference: <https://github.com/ananian04/DozerNet> (branch `main`)

Backend files to view & recreate

Path on the reference repo: `src/main/java/com/dozernet/module2_booking/<path>` — create the same file at `module2_booking/<path>` in your own branch.

- `CancellationPolicy.java`
- `SriLankaDistricts.java`
- `dto/BookingForm.java`
- `dto/ReturnInspectionForm.java`
- `entity/Booking.java`
- `entity/BookingStatus.java`
- `entity/MachineReplacementLog.java`
- `entity/ResponsibleParty.java`
- `entity/ReturnInspection.java`
- `repository/BookingRepository.java`
- `repository/MachineReplacementLogRepository.java`
- `repository/ReturnInspectionRepository.java`
- `service/BookingService.java`
- `service/ReturnInspectionService.java`
- `web/AdminBookingController.java`
- `web/CustomerBookingController.java`

Frontend files to view & recreate

Path on the reference repo: `src/main/resources/templates/booking/<file>`.

- `admin-bookings.html`
- `admin-inspections.html`
- `admin-replace-machine.html`
- `admin-return-inspection.html`
- `my-bookings.html`
- `new.html`

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.

PART B — #3 Fleet (JCB) Management

Jayawardhana A.G.D.L. · IT25100798

Branch: `feature/m3-fleet-management` | Your package: `com.dozernet.module3_fleet` | Your templates: `templates/fleet/`

Nothing has been pushed to `feature/m3-fleet-management` yet beyond the shared scaffold. Open the reference repo below, copy each file's contents, and create it at the matching path in your own branch.

Reference: <https://github.com/ananian04/DozerNet> (branch `main`)

Backend files to view & recreate

Path on the reference repo: `src/main/java/com/dozernet/module3_fleet/<path>` — create the same file at `module3_fleet/<path>` in your own branch.

- `dto/MachineForm.java`
- `dto/OwnerRegisterForm.java`
- `entity/Machine.java`
- `entity/MachineStatus.java`
- `entity/MachineType.java`
- `entity/Ownership.java`
- `repository/MachineRepository.java`
- `service/FleetService.java`
- `web/AdminFleetController.java`
- `web/MachineCatalogController.java`
- `web/OwnerController.java`

Frontend files to view & recreate

Path on the reference repo: `src/main/resources/templates/fleet/<file>`.

- `admin-listings.html`
- `admin-machine-form.html`
- `admin-machines.html`
- `catalog.html`
- `detail.html`
- `owner-dashboard.html`
- `owner-machine-form.html`
- `owner-register.html`

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.

PART B — #4 Operator Management

Perera M.A.S.A. · IT25103606

Branch: feature/m4-operator-management | Your package: com.dozernet.module4_operator | Your templates: templates/operator/

Nothing has been pushed to feature/m4-operator-management yet beyond the shared scaffold. Open the reference repo below, copy each file's contents, and create it at the matching path in your own branch.

Reference: <https://github.com/ananian04/DozerNet> (branch main)

Backend files to view & recreate

Path on the reference repo: src/main/java/com/dozernet/module4_operator/<path> — create the same file at module4_operator/<path> in your own branch.

- dto/CompanyOperatorForm.java
- dto/DriverRegisterForm.java
- entity/Assignment.java
- entity/JobStatus.java
- entity/OperatorProfile.java
- repository/AssignmentRepository.java
- repository/OperatorProfileRepository.java
- service/OperatorService.java
- web/AdminOperatorController.java
- web/DriverRegistrationController.java
- web/OperatorController.java

Frontend files to view & recreate

Path on the reference repo: src/main/resources/templates/operator/<file>.

- admin-assignments.html
- admin-operators.html
- dashboard.html
- driver-register.html

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.

PART B — #5 Maintenance & Service Management

Ruvendi R.G.K. · IT25101701

Branch: `feature/m5-maintenance-service` | Your package: `com.dozernet.module5_maintenance` | Your templates: `templates/maintenance/`

Nothing has been pushed to `feature/m5-maintenance-service` yet beyond the shared scaffold. Open the reference repo below, copy each file's contents, and create it at the matching path in your own branch.

Reference: <https://github.com/ananian04/DozerNet> (branch `main`)

Backend files to view & recreate

Path on the reference repo: `src/main/java/com/dozernet/module5_maintenance/<path>` — create the same file at `module5_maintenance/<path>` in your own branch.

- `dto/ScheduleForm.java`
- `entity/MaintenanceRecord.java`
- `entity/MaintenanceStatus.java`
- `repository/MaintenanceRecordRepository.java`
- `service/MaintenanceService.java`
- `web/AdminMaintenanceController.java`

Frontend files to view & recreate

Path on the reference repo: `src/main/resources/templates/maintenance/<file>`.

- `admin-maintenance.html`
- `history.html`

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.

PART B — #6 Payment & Administration

Thathsara J.P.T. · IT25103683

Branch: `feature/m6-payment-admin` | Your package: `com.dozernet.module6_payment` | Your templates: `templates/payment/`

Nothing has been pushed to `feature/m6-payment-admin` yet beyond the shared scaffold. Open the reference repo below, copy each file's contents, and create it at the matching path in your own branch.

Reference: <https://github.com/ananian04/DozerNet> (branch `main`)

Backend files to view & recreate

Path on the reference repo: `src/main/java/com/dozernet/module6_payment/<path>` — create the same file at `module6_payment/<path>` in your own branch.

- `entity/Invoice.java`
- `entity/InvoiceStatus.java`
- `entity/Payment.java`
- `entity/PaymentMethod.java`
- `pricing/PricingBreakdown.java`
- `pricing/PricingSelector.java`
- `pricing/PricingStrategy.java`
- `pricing/StandardPricingStrategy.java`
- `pricing/TransportSurchargeCalculator.java`
- `report/ReportExporter.java`
- `report/ReportService.java`
- `report/ReportTable.java`
- `repository/InvoiceRepository.java`
- `repository/PaymentRepository.java`
- `service/AdminService.java`
- `service/CardPaymentRequest.java`
- `service/PaymentService.java`
- `web/AdminAuditController.java`
- `web/AdminDashboardController.java`
- `web/AdminInvoiceController.java`
- `web/AdminReportController.java`
- `web/AdminUserController.java`
- `web/CustomerInvoiceController.java`

Frontend files to view & recreate

Path on the reference repo: `src/main/resources/templates/payment/<file>`.

- `../admin/dashboard.html` (note: `templates/admin/`, not `templates/payment/`)
- `admin-audit.html`
- `admin-invoice-detail.html`
- `admin-invoices.html`
- `admin-reports.html`
- `admin-users.html`
- `customer-invoice-detail.html`
- `customer-invoices.html`
- `customer-pay.html`
- `customer-thank-you.html`

Viva tip: you should be able to explain every file in your module line by line — what business rule it enforces, which repository/service it talks to, and why. That's what's graded as your individual contribution.